Exercise 1 — Bank Customer Queue System (Queue / FIFO)

Requirements: Create a `Customer` class with the attributes: - customerId - name - serviceType (e.g. “Deposit”, “Withdrawal”, “Loan”) - arrivalTime

Use `Queue<Customer>` to simulate the customer queue at a bank counter: 1. Add a new customer to the queue. 2. Serve (dequeue) the customer at the front of the queue. 3. View the next customer waiting. 4. Display the full list of customers currently waiting.

Requirement: When serving a customer, print “Now serving: [Name] - Service: [serviceType]”

Exercise 2 — Customer Support Ticket System (PriorityQueue)

Requirements: Create a `SupportTicket` class with: - ticketId - customerName - issueDescription - priority (1 = highest urgency, 5 = lowest) - status

Use `PriorityQueue` to automatically order tickets by priority (lower priority number is handled first).

Functions to implement: 1. Create a new ticket. 2. Process the ticket with the highest priority. 3. Display the list of waiting tickets (in priority order). 4. Show a count of tickets at each priority level.

Exercise 3 — Reversing a String with a Stack

Write a function that uses a stack to reverse a given string.

- Sample Input: “hello”
- Expected Output: “olleh”

Exercise 4 — Task Management Queue

Implement a queue to simulate a task management system where tasks arrive and are processed in FIFO order.

- Write functions to enqueue new tasks, dequeue and process tasks, and view the current task at the front.

Exercise 5 — Parallel Task Processing Queue (Bonus)

Simulate a server that receives and processes requests in a queue. Implement multithreading to handle multiple tasks in parallel.

- **Bonus:** Introduce task priorities (`PriorityQueue`) so that high-priority tasks are handled first.

Exercise 6 — Balanced Parentheses Checker

Write a program that uses a **stack** to check whether the parentheses/brackets (`()`, `{}`, `[]`) in a mathematical expression are correctly opened and closed in order.

- Input: a string expression containing brackets.
- Output: `true` if the brackets are balanced, `false` otherwise.

Exercise 7 — Infix to Postfix Conversion

Write a program that converts an **Infix** expression to a **Postfix** expression using a **stack**.

- Input: an infix expression string (e.g. "A + B * (C - D)").
- Output: the corresponding postfix expression string.

Exercise 8 — Implementing a Queue Using Two Stacks

Write a program that implements a **queue** using two stacks. You need to implement two main methods: `enqueue` (add an element to the queue) and `dequeue` (remove an element from the queue).

Exercise 9 — Palindrome Number Check Using a Queue

Write a program that checks whether an integer is a palindrome number, using a **queue**.

- Input: an integer.
- Output: `true` if it is a palindrome number, `false` otherwise.

Exercise 10 — Warehouse Crate Stacking System (Capstone)

You are asked to build a warehouse management program in which crates are stacked on top of each other on **shelves**. Each shelf follows these management rules:

1. A heavier crate may not be placed on top of a lighter crate.
2. The system allows moving crates between shelves in order to sort them or take them out.

Your task — write a Java program to: 1. **Model each shelf** using a **Stack** data structure. 2. **Input crates** from the keyboard (each crate has: an item code and a weight). 3. **Sort the crates** across the shelves so that the heaviest crate ends up at the bottom of a shelf and the lightest ends up on top. 4. **Retrieve a crate** from the shelves on request (assume the user requests a crate by a specific item code).

Detailed Requirements

1. **Modeling a shelf**
   - Use a **Stack** to store the crates.
   - Each crate is an object with the attributes:

```
class Crate {
    private String id;     // item code
    private int weight;    // weight
}
```

2. **Core functions**

- **Add a crate to a shelf:**
  - Input the crate's information from the keyboard.
  - The crate is temporarily placed on a shelf in the order it was entered.
- **Sort the crates:**
  - Use an approach similar to the "Tower of Hanoi" puzzle to move crates between shelves so that heavier crates end up lower down.
  - Limit the setup to exactly **3 shelves**.
- **Retrieve a crate:**
  - Ask the user to enter the item code of the crate to retrieve.
  - Search for the crate by its code among the shelves.
  - Move crates as needed to retrieve the requested crate without violating the stacking rule.
- **Display shelf status:**
  - Print the list of crates on each shelf, from top to bottom.

3. **Sorting algorithm guidance**
   - Apply the "Tower of Hanoi" rule:
     - Shelf 1: holds the crates initially.
     - Shelf 2: a temporary shelf.
     - Shelf 3: where the sorted crates end up.
   - Move crates based on weight (heaviest ends up at the bottom).
4. **Programming requirements**
   - At minimum, use the following data structures: **Stack**, **ArrayList**.
   - Use recursion to sort the crates (hint: similar to the Tower of Hanoi algorithm).
   - Ensure good performance and clear, readable source code.
5. **Extension (bonus)**
   - Add a feature to save the shelf state to a file (`.txt`) and restore the state from that file.

Sample Output

1. **Initial shelf state:**

```
Shelf 1: [P1: 5kg, P2: 3kg, P3: 2kg]
Shelf 2: []
Shelf 3: []
```

2. **After sorting:**

```
Shelf 1: []
Shelf 2: []
Shelf 3: [P3: 2kg, P2: 3kg, P1: 5kg]
```

3. **When retrieving crate P2:**

```
Retrieve P2 from Shelf 3.
Shelf state:
Shelf 1: []
```

```
Shelf 2: []
Shelf 3: [P3: 2kg, P1: 5kg]
```

(Hint: refer to solutions for the Tower of Hanoi puzzle.)

Exercise 11 — Basic LinkedList Operations

Using `LinkedList<Integer>` in Java, implement the following operations:

- Create a `LinkedList` and add several elements to it.
- Print the elements.
- Add an element at the beginning/end of the list.
- Remove an element from the beginning/end.
- Insert an element at any given position.
- Search for an element.
- Calculate the sum of the elements in the list.
- Reverse the linked list.
- Check whether the linked list contains a cycle.
- Find the middle node of the list.
- Merge two unsorted linked lists into one sorted linked list.
- Remove duplicate elements from an already-sorted list.